Patrones de Diseño

Guía Completa y Detallada

Diseño de Software

Grado en Ingeniería Informática

Universidad de Santiago de Compostela

Curso 2025/2026

Cubre los temas:

T8. Patrones de Creación (Abstract Factory, Factory Method)

T9. Patrones Estructurales (Adapter, Bridge, Composite, Decorator, Proxy)

T10. Patrones de Comportamiento (Observer, State, Strategy, Template Method)

Este documento explica cada patrón desde cero, incluyendo cómo leer diagramas UML, cómo identificar cada patrón visualmente, cómo diferenciarlo de otros similares, y ejemplos detallados de cada uno.

0 1 Contents

1 Cómo leer un diagrama UML: guía para principiantes **4**
1.1 Qué es una clase en UML . . . 4
1.2 Las relaciones entre clases: las flechas . . . 4
1.2.1 Herencia (extends / "es un") . . . 4
1.2.2 Implementación de interfaz . . . 5
1.2.3 Asociación / Referencia ("tiene un" o "usa un") . . . 5
1.2.4 Agregación y Composición . . . 6
1.2.5 Multiplicidad: cuántos objetos hay . . . 6
1.2.6 La etiqueta «instantiate» . . . 6
1.3 Resumen visual de las flechas . . . 6

2 Introducción a los Patrones de Diseño **8**
2.1 Las tres categorías de patrones . . . 8
2.2 Principio fundamental: programar para interfaces . . . 8

3 T8: Patrones de Creación **9**
3.1 Abstract Factory (Fábrica Abstracta) . . . 9
3.1.1 ¿Qué problema resuelve? . . . 9
3.1.2 Cuándo usarlo . . . 9
3.1.3 Los participantes (las clases del patrón) . . . 10
3.1.4 La estructura UML explicada paso a paso . . . 10
3.1.5 Ejemplo del diagrama de las diapositivas explicado . . . 11
3.1.6 Ventajas e inconvenientes . . . 11
3.2 Factory Method (Método de Fabricación) . . . 13
3.2.1 ¿Qué problema resuelve? . . . 13
3.2.2 Cuándo usarlo . . . 13
3.2.3 Los participantes . . . 13
3.2.4 La estructura UML explicada . . . 14
3.2.5 Ventajas e inconvenientes . . . 15
3.2.6 Comparación: Abstract Factory vs. Factory Method . . . 15

4 T9: Patrones Estructurales **16**
4.1 Adapter (Adaptador) . . . 16
4.1.1 ¿Qué problema resuelve? . . . 16
4.1.2 Cuándo usarlo . . . 16
4.1.3 Los participantes . . . 16
4.1.4 La estructura UML explicada . . . 17
4.1.5 Ventajas e inconvenientes . . . 18
4.2 Bridge (Puente) . . . 19
4.2.1 ¿Qué problema resuelve? . . . 19
4.2.2 Cuándo usarlo . . . 19
4.2.3 Los participantes . . . 19
4.2.4 La estructura UML explicada . . . 20
4.2.5 Ventajas e inconvenientes . . . 21
4.3 Composite (Compuesto) . . . 22
4.3.1 ¿Qué problema resuelve? . . . 22

4.3.2 Cuándo usarlo . . . 22
4.3.3 Los participantes . . . 22
4.3.4 La señal más importante: la auto-referencia . . . 23
4.3.5 La estructura UML explicada . . . 23
4.3.6 Ventajas e inconvenientes . . . 24
4.4 Decorator (Decorador) . . . 25
4.4.1 ¿Qué problema resuelve? . . . 25
4.4.2 Cuándo usarlo . . . 25
4.4.3 Los participantes . . . 25
4.4.4 La estructura UML explicada . . . 26
4.4.5 Diferencias cruciales con otros patrones similares . . . 27
4.4.6 Ventajas e inconvenientes . . . 27
4.5 Proxy (Apoderado) . . . 28
4.5.1 ¿Qué problema resuelve? . . . 28
4.5.2 Tipos de Proxy . . . 28
4.5.3 Los participantes . . . 28
4.5.4 La estructura UML explicada . . . 29
4.5.5 Diferencias con Adapter y Decorator . . . 30

5 T10: Patrones de Comportamiento 31
5.1 Observer (Observador) . . . 31
5.1.1 ¿Qué problema resuelve? . . . 31
5.1.2 Cuándo usarlo . . . 31
5.1.3 Los participantes . . . 31
5.1.4 La estructura UML explicada . . . 32
5.1.5 Ventajas e inconvenientes . . . 33
5.2 State (Estado) . . . 34
5.2.1 ¿Qué problema resuelve? . . . 34
5.2.2 Cuándo usarlo . . . 34
5.2.3 Los participantes . . . 34
5.2.4 La estructura UML explicada . . . 35
5.2.5 Implementación: ¿quién define las transiciones? . . . 36
5.2.6 Ventajas e inconvenientes . . . 36
5.3 Strategy (Estrategia) . . . 37
5.3.1 ¿Qué problema resuelve? . . . 37
5.3.2 Cuándo usarlo . . . 37
5.3.3 Los participantes . . . 37
5.3.4 La estructura UML explicada . . . 38
5.3.5 La diferencia crucial: State vs. Strategy . . . 38
5.3.6 Ventajas e inconvenientes . . . 39
5.4 Template Method (Método Plantilla) . . . 40
5.4.1 ¿Qué problema resuelve? . . . 40
5.4.2 Cuándo usarlo . . . 40
5.4.3 Los participantes . . . 40
5.4.4 La estructura UML explicada . . . 41
5.4.5 Los tres tipos de operaciones en el método plantilla . . . 42
5.4.6 Template Method vs. Strategy . . . 42
5.4.7 Ventajas e inconvenientes . . . 42

6 Tablas Resumen: Identificación Rápida **43**
6.1 Señales visuales en UML por patrón 43
6.2 Comparativa completa de todos los patrones 43
6.3 Los tríos confusos: Adapter, Proxy y Decorator 44
6.4 State vs. Strategy vs. Template Method 44
6.5 Abstract Factory vs. Factory Method 45

7 Guía Práctica para el Examen **46**
7.1 Método en 5 pasos para identificar un patrón 46
7.2 Preguntas tipo que suelen aparecer 46
7.3 Participantes rápidos de cada patrón 46

1 1 Cómo leer un diagrama UML: guía para principiantes

Antes de estudiar los patrones, es **imprescindible** entender cómo se leen los diagramas UML (Unified Modeling Language). UML es simplemente un lenguaje visual para representar clases, objetos y cómo se relacionan entre sí. Aquí explicamos todo desde cero.

1.1 Qué es una clase en UML

Una clase en UML se dibuja como un rectángulo dividido en tres partes:

Estructura de una clase UML

```
+---------------------+
|   NombreDeLaClase   |  <- Nombre (si esta en cursiva, es abstracta)
+---------------------+
| -atributo1 : Tipo   |  <- Atributos (variables internas)
| -atributo2 : Tipo   |
+---------------------+
| +metodo1()          |  <- Metodos (funciones)
| +metodo2(param)     |
+---------------------+
```

Significado de los símbolos de visibilidad

- `+` delante de un método o atributo = **público** (cualquiera puede acceder)
- `-` delante de un método o atributo = **privado** (solo la propia clase)
- `#` delante = **protegido** (la clase y sus subclases)
- Si el nombre de la clase o un método está en *cursiva*, significa que es **abstracto** (no tiene implementación concreta, obliga a las subclases a implementarlo)

1.2 Las relaciones entre clases: las flechas

Las flechas son la parte más importante. Cada tipo de flecha significa una cosa diferente.

1.2.1 Herencia (extends / "es un")

Flecha de herencia

```
ClasePadre
    ^
    |  (flecha solida con triangulo hueco apuntando al padre)
    |
ClaseHija
```

Herencia en la vida real

Imagina que tienes una clase `Animal` y una clase `Perro`. Un `Perro` **es un** `Animal`. La flecha de herencia va de `Perro` hacia `Animal`. Esto significa que `Perro` hereda todos los métodos y atributos de `Animal`, y puede añadir los suyos propios o redefinir los existentes.
Si la clase padre es *abstracta* (nombre en cursiva), significa que no se puede crear directamente un objeto de esa clase. Solo se pueden crear objetos de las clases hija.

1.2.2 Implementación de interfaz

Flecha de implementación

```
<<interface>>
IAnimal
    ^
    | (flecha DISCONTINUA con triangulo hueco)
    |
Perro
```

Interfaz vs Clase Abstracta

Una **interfaz** (marcada con `«interface»`) es como un contrato: dice *qué* métodos debe tener una clase, pero no *cómo* los implementa. Es como decir "todo lo que sea un Animal debe poder moverse y comer", sin decir cómo lo hace.
La diferencia con una clase abstracta es que la clase abstracta puede tener ALGUNA implementación, mientras que la interfaz pura no tiene ninguna.

1.2.3 Asociación / Referencia ("tiene un" o "usa un")

Flecha de asociación

```
ClaseA ---------> ClaseB
       -nombreRef
```

Referencia entre clases

Imagina que `Coche` tiene una referencia a `Motor`. El coche "tiene un" motor. La flecha va de `Coche` a `Motor`. Debajo de la flecha suele aparecer el nombre de la variable: `-motor`.
Si la flecha es **sólida con punta de flecha normal**: asociación simple (tiene una referencia). Si la flecha es **discontinua**: dependencia (usa temporalmente, por ejemplo como parámetro).

1.2.4 Agregación y Composición

Agregación y Composición

```
Composicion:    ClaseA <>-----------> ClaseB   (rombo relleno)
Agregacion:     ClaseA <> - - - - -> ClaseB   (rombo hueco)
```

Diferencia agregación/composición

Composición (rombo relleno): la parte no puede existir sin el todo. Si destruyes el todo, destruyes las partes. Ejemplo: una `Casa` compuesta de `Habitaciones`. Si destruyes la casa, las habitaciones dejan de existir.
Agregación (rombo hueco): la parte puede existir independientemente del todo. Ejemplo: un `Equipo` formado por `Jugadores`. Si el equipo desaparece, los jugadores siguen existiendo.

1.2.5 Multiplicidad: cuántos objetos hay

Al lado de las flechas aparecen números que indican cuántos objetos hay en cada extremo:

Multiplicidad

```
ClaseA 1 ---------> * ClaseB
```

- `1` = exactamente uno
- `*` = muchos (cero o más)
- `1..*` = uno o más
- `0..1` = cero o uno (opcional)

En el ejemplo: un objeto de ClaseA se asocia con muchos (`*`) objetos de ClaseB.

1.2.6 La etiqueta «instantiate»

En los diagramas de patrones de diseño verás flechas discontinuas con la etiqueta `«instantiate»`. Esto significa que una clase **crea instancias** (es decir, crea objetos) de otra clase. Es como decir "esta clase hace `new OtraClase()`".

1.3 Resumen visual de las flechas

Guía rápida de flechas UML

```
Herencia:        A ------> B    A extiende B (solida, triangulo hueco)
Implementacion:  A - - - -> B  A implementa la interfaz B (discontinua, triangulo)
Asociacion:      A --------> B    A tiene referencia a B (solida, punta flecha)
Dependencia:     A - - - -> B   A usa B temporalmente (discontinua, punta flecha)
Composicion:     A <>---- B     A contiene B (rombo relleno)
```

```
Agregacion:      A <>---- B     A agrupa B (rombo vacio)
Instanciacion:   A - - -"instantiate"- -> B  A crea objetos de B
```

2 1 Introduc a los Patrones de Diseño

¿Qué es un patrón de diseño?

Un **patrón de diseño** es una solución reutilizable a un problema recurrente en el diseño de software orientado a objetos. No es código concreto, sino una plantilla o receta que describe cómo resolver un problema de diseño.
Los patrones fueron popularizados por el libro de Gamma, Helm, Johnson y Vlissides (conocidos como "Gang of Four" o GoF): *Design Patterns. Elements of Reusable Object-Oriented Software* (1994).

2.1 Las tres categorías de patrones

azulclaro **Categoría**	**¿Qué hace?**
Creación	Abstraen y controlan **cómo se crean los objetos**. Ocultan qué clases concretas se instancian.
Estructurales	Tratan la **composición de clases y objetos** para formar estructuras más grandes.
Comportamiento	Definen cómo **interactúan y se comunican** los objetos.

2.2 Principio fundamental: programar para interfaces

Idea Clave

Todos los patrones de diseño siguen un principio básico: **programar para interfaces (o clases abstractas), no para implementaciones concretas**.
¿Qué significa esto? En lugar de que tu código dependa de clases concretas (que pueden cambiar), depende de interfaces o clases abstractas (contratos estables). Así, puedes cambiar la implementación sin tocar el código que la usa.
Ejemplo: en lugar de tener una variable de tipo `VentanaKDE`, tienes una variable de tipo `IVentana`. Así, mañana puedes poner `VentanaGNOME` sin cambiar nada más.

3 1 T8: Patrones de Creación

¿Qué son los Patrones de Creación?

Los patrones de creación **abstraen el proceso de creación de objetos**. El resto del sistema no sabe exactamente qué clase concreta se está creando; solo conoce los objetos a través de sus interfaces. Esto hace posible cambiar las clases concretas sin afectar al resto del sistema.

3.1 Abstract Factory (Fábrica Abstracta)

3.1.1 ¿Qué problema resuelve?

Imagina que estás desarrollando un toolkit de interfaz gráfica que debe funcionar en dos entornos: **KDE** y **GNOME** (dos escritorios de Linux). En ambos, necesitas crear ventanas y botones. El problema es que las ventanas y botones de KDE son diferentes a los de GNOME visualmente y en comportamiento.

Sin ningún patrón, tu código estaría lleno de comprobaciones como:

```
if (entorno == "KDE") {
    ventana = new VentanaKDE();
    boton = new BotonKDE();
} else if (entorno == "GNOME") {
    ventana = new VentanaGNOME();
    boton = new BotonGNOME();
}
```

Este código tiene varios problemas:

- Si añades un nuevo entorno (p.ej. XFCE), tienes que buscar y modificar todos los `if` del proyecto.
- El código cliente conoce todas las clases concretas, lo que crea mucho acoplamiento.
- Es fácil mezclar objetos de familias distintas (un botón KDE con una ventana GNOME).

Solución: Abstract Factory

Proporciona una **interfaz para crear familias de objetos relacionados** sin especificar sus clases concretas. El cliente solo habla con la fábrica abstracta: le pide una ventana, le pide un botón, y la fábrica concreta (KDE o GNOME) se encarga de crear los objetos correctos.

3.1.2 Cuándo usarlo

- Cuando quieres **configurar un sistema escogiendo una familia de productos** de entre varias.

- Cuando cada familia está diseñada para **usarse en conjunto** (no mezclar KDE con GNOME).
- Para proporcionar **bibliotecas de productos** de los que solo se revelan sus interfaces.

3.1.3 Los participantes (las clases del patrón)

Participantes de Abstract Factory

- **FábricaAbstracta**: es la interfaz principal. Declara los métodos de creación (p.ej. `crearVentana()`, `crearBoton()`). No dice cómo crear, solo qué se puede crear.
- **FábricaConcreta** (ej: FabricaKDE, FabricaGNOME): implementa la FábricaAbstracta y crea objetos de una familia específica.
- **ProductoAbstracto** (ej: IVentana, IBoton): interfaz para cada tipo de producto. Define qué puede hacer cualquier ventana o botón.
- **ProductoConcreto** (ej: VentanaKDE, BotonGNOME): implementa el ProductoAbstracto para una familia concreta.
- **Cliente**: usa solo la FábricaAbstracta y los ProductosAbstractos. No conoce las clases concretas.

3.1.4 La estructura UML explicada paso a paso

Estructura del patrón Abstract Factory

```
                        <<interface>>
                      IFabricaControles          Cliente
                      +crearVentana()       <--------+
                      +crearBoton()           -fabrica
                            ^
                   ---------|---------
                  |                   |
            FabricaGNOME         FabricaKDE
            +crearVentana()      +crearVentana()
            +crearBoton()        +crearBoton()
              |   |                |    |
     <<instantiate>> |      <<instantiate>>  |
                 |  |                |    |
    VentanaGNOME <-  BotonGNOME  VentanaKDE  BotonKDE
         |                              |
         v                              v
    <<interface>>                  <<interface>>
      IVentana                       IBoton
```

Cómo leer este diagrama paso a paso:

1. **IFabricaControles** es la fábrica abstracta (interfaz). Tiene dos métodos: `crearVentana()` y `crearBoton()`. Es el contrato que deben cumplir todas las fábricas.

2. **FabricaKDE** y **FabricaGNOME** implementan `IFabricaControles` (flechas discontinuas con triángulo hacia arriba). Cada una sabe cómo crear los objetos de su propia familia.

3. Las flechas `«instantiate»` muestran qué clase concreta crea cada fábrica:
   - FabricaKDE crea `VentanaKDE` y `BotonKDE`.
   - FabricaGNOME crea `VentanaGNOME` y `BotonGNOME`.

4. **IVentana** e **IBoton** son los productos abstractos. `VentanaKDE`, `VentanaGNOME` implementan `IVentana`; `BotonKDE` y `BotonGNOME` implementan `IBoton`.

5. El **Cliente** tiene una referencia `-fabrica` a `IFabricaControles` (no a una fábrica concreta). Llama a `fabrica.crearVentana()` y obtiene un `IVentana`, sin saber si es KDE o GNOME.

3.1.5 Ejemplo del diagrama de las diapositivas explicado

El ejemplo tiene dos diagramas. Vamos a analizarlos:

Primer diagrama (sin la fábrica): Muestra solo los productos y el cliente. El cliente tiene una colección de `-ventanas` (tipo `IVentana`) y otra de `-botones` (tipo `IBoton`). Las clases `VentanaKDE` y `VentanaGNOME` implementan `IVentana`. Lo mismo con los botones. Este diagrama ilustra el problema inicial: el cliente trabaja con las interfaces, pero ¿quién crea los objetos concretos?

Segundo diagrama (con la fábrica): Añade `IFabricaControles` con dos implementaciones: `FabricaKDE` y `FabricaGNOME`. El cliente ahora tiene una referencia `-fabrica` de tipo `IFabricaControles` (entre 0 y 1 fábricas, por eso `0..1`). Las flechas `«instantiate»` muestran que `FabricaKDE` crea `VentanaKDE` y `BotonKDE`, mientras que `FabricaGNOME` crea `VentanaGNOME` y `BotonGNOME`.

Flujo de ejecución

1. Al inicio del programa, alguien decide qué entorno usar y crea `new FabricaKDE()` o `new FabricaGNOME()`.
2. Esta fábrica se le pasa al `Cliente`.
3. El Cliente llama a `fabrica.crearVentana()` → obtiene un `IVentana` (sin saber si es KDE o GNOME).
4. El Cliente llama a `fabrica.crearBoton()` → obtiene un `IBoton`.
5. El Cliente usa `ventana.maximizar()`, `boton.clicar()`, etc. mediante las interfaces.
6. Para cambiar a GNOME, solo cambia la fábrica. El resto del código queda intacto.

3.1.6 Ventajas e inconvenientes

Ventajas:

- **Desacoplamiento**: los clientes solo usan interfaces, no clases concretas.

- **Cambio de familia fácil**: basta con cambiar la fábrica concreta.
- **Consistencia**: garantiza que todos los objetos creados son de la misma familia (no mezclas KDE con GNOME).

Inconvenientes:

- **Difícil añadir nuevos productos**: si quieres añadir un nuevo tipo de objeto (p.ej. un menú), tienes que modificar la interfaz `IFabricaControles` y todas las fábricas concretas.

3.2 Factory Method (Método de Fabricación)

3.2.1 ¿Qué problema resuelve?

Imagina un framework de aplicaciones que puede gestionar distintos tipos de documentos. La clase `Aplicacion` define el flujo general: cómo abrir, cerrar, guardar documentos. Pero `Aplicacion` no sabe qué tipo concreto de documento va a manejar (¿un PDF? ¿un Word? ¿un documento propio?).

Sin Factory Method, tendríamos que modificar `Aplicacion` cada vez que añadamos un tipo de documento. Eso viola el principio de "abierto para extensión, cerrado para modificación".

Solución: Factory Method

Define una interfaz para crear un objeto, pero **deja que las subclases decidan qué clase instanciar**. La clase base declara el método de creación como abstracto. Las subclases lo implementan. El método de creación se llama **método de fabricación** (factory method).

3.2.2 Cuándo usarlo

- Cuando una clase **no puede prever la clase** de los objetos que debe crear.
- Cuando quieres que las **subclases decidan** qué objetos crear.
- Para proporcionar a subclases un **enganche** para una versión extendida de un objeto.

3.2.3 Los participantes

Participantes de Factory Method

- **Producto**: define la interfaz de los objetos que crea el método de fabricación. Es la clase base abstracta (o interfaz) de los objetos que se crearán.
- **ProductoConcreto**: implementa la interfaz Producto. Es el objeto real que se crea.
- **Creador**: clase abstracta que declara el método de fabricación (*abstracto*). Puede tener otros métodos que usan el producto sin saber cuál es exactamente.
- **CreadorConcreto**: subclase concreta del Creador. Redefine el método de fabricación y devuelve un ProductoConcreto específico.

3.2.4 La estructura UML explicada

Estructura del patrón Factory Method

```
Documento (Producto)            Aplicacion (Creador)
+abrir()                        +crearDocumento()  <- metodo abstracto
+cerrar()                       +nuevoDocumento()
+guardar()                      +abrirDocumento()
+deshacer()
     ^                                  ^
     |                                  |
MiDocumento              MiAplicacion (CreadorConcreto)
(ProductoConcreto)       +crearDocumento()  -> return new MiDocumento
     ^                           |
     |    <<instantiate>>        |
     +---------------------------+

Lo que hace nuevoDocumento() en Aplicacion:
-------------------------------------------
Documento doc = crearDocumento();  <- llama al factory method
docs.add(doc);
doc.abrir();
```

Cómo leer este diagrama:

1. `Documento` es el Producto (clase abstracta con métodos como `abrir()`, `cerrar()`...).
2. `MiDocumento` es el ProductoConcreto: extiende `Documento`.
3. `Aplicacion` es el Creador (clase abstracta). Tiene el método abstracto `crearDocumento()` y métodos concretos como `nuevoDocumento()` que **usan** el factory method sin saber qué tipo de documento crea.
4. `MiAplicacion` es el CreadorConcreto: extiende `Aplicacion` e implementa `crearDocumento()` devolviendo `new MiDocumento()`.
5. La flecha «`instantiate`» de `MiAplicacion` a `MiDocumento` muestra quién crea a quién.

Flujo de ejecución

1. Se crea `MiAplicacion app = new MiAplicacion()`.
2. Se llama a `app.nuevoDocumento()`.
3. Dentro de `nuevoDocumento()`, se llama a `crearDocumento()` (el factory method).
4. Como `app` es un `MiAplicacion`, se ejecuta el `crearDocumento()` de `MiAplicacion`, que devuelve `new MiDocumento()`.
5. El resultado se guarda como `Documento doc` (tipo abstracto), y se llama a `doc.abrir()`.
6. Si mañana añadimos `OtraAplicacion` con `crearDocumento()` devolviendo `new OtroDocumento()`, el método `nuevoDocumento()` funciona sin cambios.

3.2.5 Ventajas e inconvenientes

Ventajas:

- Los clientes solo tratan con la **interfaz Producto**: no dependen de clases concretas.
- **Dota a subclases de un enganche**: una subclase puede proporcionar una versión extendida de un objeto.
- Puede **conectar jerarquías paralelas**: tanto la jerarquía de Creadores como la de Productos pueden crecer independientemente.

Inconvenientes:

- Los clientes pueden **verse obligados a heredar** de la clase Creador solo para crear un ProductoConcreto diferente.

3.2.6 Comparación: Abstract Factory vs. Factory Method

Abstract Factory vs. Factory Method

azulclaro **Aspecto**	**Factory Method**	**Abstract Factory**
Mecanismo	Herencia (subclase redefine)	Composición (objeto fábrica)
¿Qué crea?	Un solo producto	Familia de productos
¿Quién decide?	La subclase Creador	El objeto FábricaConcreta
Complejidad	Menor	Mayor
Cambio en ejecución	No (la subclase es fija)	Sí (se puede cambiar la fábrica)
Señal UML	Clase abstracta con método abstracto que devuelve un objeto	Interfaz con varios métodos crearX() y varias clases fábrica

Regla rápida: si ves una clase abstracta con un único método de creación abstracto → Factory Method. Si ves una interfaz fábrica con múltiples métodos de creación y varias familias de productos → Abstract Factory.

4 1 T9: Patrones Estructurales

¿Qué son los Patrones Estructurales?

Los patrones estructurales tratan la **composición de clases y objetos** para formar estructuras más grandes y flexibles. Se preguntan: ¿cómo ensamblo las clases para construir algo más complejo?

4.1 Adapter (Adaptador)

4.1.1 ¿Qué problema resuelve?

Tienes un sistema que trabaja con una interfaz bien definida (`IForma` con métodos `cajaLimitrofe()` y `crearManipulador()`). Ahora quieres reutilizar una clase de terceros (`VistaTexto`) que ofrece funcionalidad útil (`obtenerExtension()`) pero su interfaz no coincide con la que espera tu sistema.

No puedes modificar `VistaTexto` (es de terceros o es código legacy). ¿Qué haces?

Solución: Adapter

Creas una clase **Adaptador** que **implementa la interfaz que espera tu sistema** y **delega internamente** en el objeto que tiene la interfaz incompatible. Es como el enchufe de viaje: el adaptador tiene la forma que necesita el enchufe de la pared (interfaz objetivo), pero internamente conecta con el dispositivo que tiene otra forma (adaptado).

4.1.2 Cuándo usarlo

- Cuando quieres usar una **clase existente cuya interfaz no encaja** con la que necesitas.
- Para crear una **clase reutilizable que coopere** con clases no previstas.
- Cuando no es práctico heredar de cada clase existente para adaptar su interfaz.

4.1.3 Los participantes

Participantes de Adapter

- **Objetivo** (Target): la interfaz que espera el Cliente. Es el "contrato" que debe cumplir el Adaptador.
- **Cliente**: usa objetos que implementan la interfaz Objetivo.
- **Adaptado** (Adaptee): la clase existente con interfaz incompatible que quieres reutilizar.

- **Adaptador** (Adapter): implementa la interfaz Objetivo y tiene una referencia al Adaptado. Traduce las llamadas del Objetivo en llamadas al Adaptado.

4.1.4 La estructura UML explicada

Estructura del patrón Adapter

```
                 <<interface>>
                   IForma                        VistaTexto (Adaptado)
                   +cajaLimitrofe()              +obtenerExtension()
                   +crearManipulador()
                        ^
          ---------------|---------------
        |                                |
     Linea                        FormaTexto (Adaptador)
     (implementacion normal)       +cajaLimitrofe()
                                   +crearManipulador()
                                        |
                     -texto (referencia a VistaTexto)
                                        |
                                        v
                                texto.obtenerExtension()

   EditorDibujo (Cliente) -------> IForma (Objetivo)
```

Cómo leer este diagrama:

1. `IForma` es el Objetivo: la interfaz que espera el `EditorDibujo` (Cliente).
2. `Linea` implementa `IForma` directamente (funciona sin adaptador).
3. `VistaTexto` es el Adaptado: tiene `obtenerExtension()` pero no implementa `IForma`.
4. `FormaTexto` es el Adaptador:
   - **Implementa** `IForma` (flecha discontinua hacia arriba a IForma).
   - **Tiene una referencia** `-texto` a `VistaTexto` (flecha hacia VistaTexto).
   - En `cajaLimitrofe()`: llama a `texto.obtenerExtension()` y traduce el resultado.
   - El cliente (`EditorDibujo`) ve `FormaTexto` como cualquier otra `IForma`.

Cómo funciona la traducción de llamadas

El Cliente llama a `forma.cajaLimitrofe()` sobre un objeto `FormaTexto`.
El `FormaTexto` internamente hace:

```
public Rectangle cajaLimitrofe() {
    Point ext = texto.obtenerExtension();  // llama al
       adaptado
    return new Rectangle(0, 0, ext.x, ext.y);  // traduce el
```

```
        resultado
}
```

El cliente no sabe nada de `VistaTexto` ni de `obtenerExtension()`.

4.1.5 Ventajas e inconvenientes

Ventajas:

- El mismo Adaptador puede funcionar con muchos Adaptados (si comparten comportamiento).
- Permite **reutilizar código existente** sin modificarlo.

Inconvenientes:

- Es más difícil redefinir el comportamiento del Adaptado (hay que crear subclases).

4.2 Bridge (Puente)

4.2.1 ¿Qué problema resuelve?

Imagina que tienes ventanas de dos tipos: `VentanaIcono` y `VentanaFlotante`. Y además tienes dos plataformas: KDE y GNOME. Si usas herencia clásica, necesitas:

- `VentanaIconoKDE`
- `VentanaIconoGNOME`
- `VentanaFlotanteKDE`
- `VentanaFlotanteGNOME`

Eso son 4 clases para 2 tipos $\times$ 2 plataformas. Si añades una plataforma más (XFCE) y un tipo más (VentanaDiálogo), necesitarías $3 \times 3 = 9$ clases. Esto es la "explosión de subclases".

Solución: Bridge

Separa las dos dimensiones de variación en **dos jerarquías independientes**:
- La **abstracción**: los tipos de ventana (VentanaIcono, VentanaFlotante).
- La **implementación**: las plataformas (KDE, GNOME).

Las conecta mediante una referencia (el "puente"): la abstracción **tiene una referencia** al implementador. Ahora con N tipos y M plataformas solo necesitas $N+M$ clases en lugar de $N \times M$.

4.2.2 Cuándo usarlo

- Para **evitar un vínculo permanente** entre abstracción e implementación.
- Cuando abstracción e implementación deban **ampliarse independientemente**.
- Para que los cambios en la implementación no afecten a los clientes.

4.2.3 Los participantes

Participantes de Bridge

- **Abstraccion**: define la interfaz de la abstracción y mantiene una referencia a un objeto Implementador (`-imp`).
- **AbstraccionRefinada**: extiende la interfaz de Abstraccion (p.ej. VentanaIcono, VentanaFlotante).
- **Implementador**: define la interfaz para las clases de implementación (operaciones de bajo nivel).
- **ImplementadorConcreto**: implementa la interfaz Implementador (p.ej. VentanaImpKDE, VentanaImpGNOME).

4.2.4 La estructura UML explicada

Estructura del patrón Bridge

```
 JERARQUIA DE ABSTRACCION          JERARQUIA DE IMPLEMENTACION
 ------------------------          ------------------------------
       Ventana                            VentanaImp
   +dibujarTexto()    -imp --------->  +dispDibujarTexto()
   +dibujarRect()                      +dispDibujarLinea()
        ^                                     ^
   _____|_____                         _______|_______
  |           |                       |               |
VentanaIcono  VentanaFlotante    VentanaImpKDE    VentanaImpGNOME
+dibujarBorde() +dibujarBorde()   +dispDibujarTexto() +dispDibujarTexto()
                                  +dispDibujarLinea() +dispDibujarLinea()

Codigo de Ventana.dibujarRect():
  imp.dispDibujarLinea(tl, tr);
  imp.dispDibujarLinea(tr, br);
  imp.dispDibujarLinea(br, bl);
  imp.dispDibujarLinea(bl, tl);
```

Cómo leer este diagrama:

1. Hay **dos jerarquías separadas**: a la izquierda los tipos de ventana, a la derecha las implementaciones de plataforma.
2. `Ventana` (la abstracción) tiene una referencia `-imp` a `VentanaImp` (el implementador). Esta es la flecha de agregación que forma el "puente".
3. `VentanaIcono` y `VentanaFlotante` son subclases de `Ventana` (herencia normal).
4. `VentanaImpKDE` y `VentanaImpGNOME` son subclases de `VentanaImp`.
5. Cuando `Ventana.dibujarRect()` necesita dibujar algo, llama a métodos del implementador: `imp.dispDibujarLinea(...)`. El método `imp` puede ser KDE o GNOME, da igual.
6. Para añadir XFCE: solo creas `VentanaImpXFCE` implementando `VentanaImp`. No tocas nada de la jerarquía de abstracción.

Flexibilidad del Bridge

- Puedes tener una `VentanaIcono` usando `VentanaImpKDE`.
- Puedes cambiar en tiempo de ejecución el implementador: la misma `VentanaIcono` puede pasar de usar KDE a usar GNOME.
- Puedes añadir un nuevo tipo de ventana sin tocar las implementaciones.
- Puedes añadir una nueva plataforma sin tocar los tipos de ventana.

4.2.5 Ventajas e inconvenientes

Ventajas:

- **Desacopla** interfaz e implementación: pueden variar independientemente.
- La implementación puede **cambiarse en tiempo de ejecución**.
- Elimina dependencias de compilación.

Inconvenientes:

- Mayor complejidad inicial en el diseño.

Adapter vs. Bridge

- **Adapter**: se usa **después de diseñar**, para resolver incompatibilidades entre interfaces ya existentes. El objetivo es hacer que funcione algo que no encajaba.
- **Bridge**: se usa **antes de diseñar** (o desde el principio), para separar dos dimensiones de variación y que puedan evolucionar independientemente.
- En palabras simples: Adapter es un parche. Bridge es un diseño planificado.

4.3 Composite (Compuesto)

4.3.1 ¿Qué problema resuelve?

En un editor gráfico, tienes objetos simples (una línea, un rectángulo, un texto) y objetos compuestos (un dibujo formado por varias líneas, rectángulos y textos). El problema es que el dibujo también puede contener otros dibujos.

Sin Composite, el código que maneja estos objetos necesita saber en todo momento si está tratando con un objeto simple o compuesto:

```
if (objeto instanceof Dibujo) {
    for each hijo in objeto.hijos:
        dibujar(hijo);  // recursivo
} else {
    objeto.dibujar();  // simple
}
```

Esto es complicado y hay que repetirlo en muchos lugares.

Solución: Composite

Trata a individuos y composiciones de forma uniforme. Tanto los objetos simples (hojas) como los compuestos implementan la misma interfaz. Cuando llamas a `dibujar()` sobre un compuesto, este itera sobre sus hijos y llama a `dibujar()` en cada uno. El cliente no necesita saber si está tratando con un simple o un compuesto.

4.3.2 Cuándo usarlo

- Para representar **jerarquías parte-todo** (un todo formado por partes que pueden ser a su vez todos).
- Para que los **clientes ignoren la diferencia** entre objetos individuales y compuestos.

4.3.3 Los participantes

Participantes de Composite

- **Componente**: interfaz común para todos los objetos (tanto simples como compuestos). Define la operación principal (`dibujar()`) y las operaciones de gestión de hijos (`añadir()`, `eliminar()`, `obtenerHijo()`).
- **Hoja**: objeto simple sin hijos. Implementa el Componente. Es el caso base de la recursión.
- **Compuesto**: objeto que puede tener hijos. Implementa el Componente Y tiene una colección de hijos (que son Componentes). Su operación itera sobre los hijos.
- **Cliente**: usa objetos a través de la interfaz Componente, sin distinguir hojas de compuestos.

4.3.4 La señal más importante: la auto-referencia

La señal clave de Composite en UML

La señal más característica de Composite en un diagrama UML es que **el Compuesto implementa la interfaz Componente Y al mismo tiempo tiene una colección de objetos de tipo Componente**.
Es decir, la interfaz se referencia a sí misma de forma recursiva: `Compuesto implements Componente` y `Compuesto has-many Componente`.

4.3.5 La estructura UML explicada

Estructura del patrón Composite

```
                    <<interface>>
                     IGrafico
                   +dibujar()
                   +trasladar()
                   +anadir(g: IGrafico)
                   +eliminar(g: IGrafico)
                   +obtenerHijo(i: int)
                          ^
            ---------------|---------------
           |               |       |       |
       Linea         Rectangulo  Texto   Dibujo
      +dibujar()    +dibujar() +dibujar() +dibujar()
      +trasladar()  ...        ...        +anadir(g)
      (sin hijos)   (sin hijos)(sin hijos)+eliminar(g)
                                          -graficos [0..*]: IGrafico
                                          |
                                for g in graficos: g.dibujar()
```

Cómo leer este diagrama:

1. `IGrafico` es el Componente: tiene `dibujar()`, `trasladar()`, y los métodos de gestión de hijos.
2. `Linea`, `Rectangulo` y `Texto` son Hojas: implementan `IGrafico` pero no tienen hijos.
3. `Dibujo` es el Compuesto: también implementa `IGrafico` (misma jerarquía), PERO además tiene `-graficos [0..*]: IGrafico` (una lista de IGrafico). Aquí está la auto-referencia.
4. Cuando llamas a `dibujo.dibujar()`, internamente hace: "para cada `g` en `graficos`: `g.dibujar()`". Si algún `g` es otro `Dibujo`, la llamada se propaga recursivamente.

Árbol de objetos Composite

Imagina esta estructura:
`d1: Dibujo`

```
+-- d2: Dibujo
|     +-- r2: Rectangulo
|     +-- l2: Linea
|     +-- t1: Texto
+-- l1: Linea
+-- r1: Rectangulo
```

Cuando el cliente llama a `d1.dibujar()`:

1. `d1` itera: llama a `d2.dibujar()`, `l1.dibujar()`, `r1.dibujar()`.
2. `d2.dibujar()` itera: llama a `r2.dibujar()`, `l2.dibujar()`, `t1.dibujar()`.
3. Cada hoja dibuja directamente.

El cliente no tuvo que saber nada de la estructura interna.

4.3.6 Ventajas e inconvenientes

Ventajas:

- Permite **agregación recursiva**: compuestos dentro de compuestos.
- El cliente **no distingue** entre hojas y compuestos.
- Facilita añadir nuevos tipos de componentes.

Inconvenientes:

- Dificulta **restringir qué componentes** pueden pertenecer a un grupo (p.ej. si quieres que un Dibujo solo contenga Líneas, hay que añadir comprobaciones en tiempo de ejecución).

4.4 Decorator (Decorador)

4.4.1 ¿Qué problema resuelve?

Tienes una clase `VistaTexto` que muestra texto. Quieres añadirle barra de desplazamiento (scroll) y un borde. Pero quieres poder hacerlo de forma opcional y combinable: a veces solo scroll, a veces solo borde, a veces ambos.

Con herencia tradicional necesitarías:

- `VistaTextoConScroll`
- `VistaTextoConBorde`
- `VistaTextoConScrollYBorde`

Si añades otra característica (sombreado, por ejemplo), necesitas el doble de clases. Esto es otro tipo de explosión de subclases.

Solución: Decorator

Envuelves el objeto original en un "decorador" que tiene la misma interfaz y añade funcionalidad antes o después de delegar en el objeto que envuelve. Puedes encadenar varios decoradores (envolver un decorador dentro de otro decorador).

4.4.2 Cuándo usarlo

- Para **añadir o quitar responsabilidades** a objetos individuales de forma dinámica.
- Cuando la extensión mediante herencia **no es viable** (demasiadas combinaciones posibles).

4.4.3 Los participantes

Participantes de Decorator

- **Componente**: interfaz base para los objetos que pueden ser decorados.
- **ComponenteConcreto**: el objeto real al que se pueden añadir responsabilidades (p.ej. `VistaTexto`).
- **Decorador**: clase abstracta. Implementa la interfaz Componente Y tiene una referencia `-componente` al mismo Componente. Delega las operaciones al componente que envuelve.
- **DecoradorConcreto**: subclase del Decorador que añade responsabilidades específicas (p.ej. `DecoradorBorde`, `DecoradorDesplazamiento`).

4.4.4 La estructura UML explicada

Estructura del patrón Decorator

```
            <<interface>>
          IComponenteVisual
            +dibujar()
            +refrescar()
                  ^
       -----------|-----------
      |                       |
  VistaTexto              Decorador
  +dibujar()              +dibujar()          <- implementa IComponenteVisual
  +refrescar()            +refrescar()
  (ComponenteConcreto)    -componente: IComponenteVisual  <- REFERENCIA A SI MISMA
                              |
                  ----------|-----------
                 |                      |
        DecoradorDesplazamiento       DecoradorBorde
        -posicion                     -anchoBorde
        +dibujar()                    +dibujar()
        +desplazar()                  +dibujarBorde()

  Decorador.dibujar():
    componente.dibujar();  <- delega en lo que envuelve

  DecoradorBorde.dibujar():
    componente.dibujar();  <- primero dibuja el componente
    dibujarBorde();        <- luego anade el borde
```

Cómo leer este diagrama:

1. `IComponenteVisual` es la interfaz base.
2. `VistaTexto` implementa `IComponenteVisual` normalmente (es el objeto real).
3. `Decorador` también implementa `IComponenteVisual` (misma interfaz) PERO tiene una referencia `-componente` que apunta a `IComponenteVisual`. Este es el punto clave: el decorador envuelve a cualquier `IComponenteVisual` (que puede ser `VistaTexto` u otro decorador).
4. `DecoradorBorde` y `DecoradorDesplazamiento` extienden `Decorador` y añaden su funcionalidad específica.

Encadenamiento de decoradores

```
// Creamos el objeto real
IComponenteVisual texto = new VistaTexto();

// Lo decoramos con scroll
IComponenteVisual conScroll = new DecoradorDesplazamiento(
```

```
   texto);

// Lo decoramos con borde encima del scroll
IComponenteVisual conScrollYBorde = new DecoradorBorde(
   conScroll);

// Al llamar a dibujar():
conScrollYBorde.dibujar();
// 1. DecoradorBorde.dibujar() -> llama a conScroll.dibujar()
// 2. DecoradorDesplazamiento.dibujar() -> llama a texto.
   dibujar()
// 3. VistaTexto.dibujar() -> dibuja el texto
// 2. (vuelve) DecoradorDesplazamiento anade su barra de
   scroll
// 1. (vuelve) DecoradorBorde anade el borde
```

4.4.5 Diferencias cruciales con otros patrones similares

Decorator vs. Composite vs. Proxy

- **Decorator vs. Composite**: ambos tienen una referencia a la misma interfaz que implementan. La diferencia: Decorator tiene **exactamente un** componente (envuelve uno) y añade responsabilidades. Composite tiene **muchos** hijos (colección) y representa jerarquías.
- **Decorator vs. Proxy**: estructuralmente son casi iguales (ambos tienen referencia al mismo tipo que implementan). La diferencia es el **propósito**: Decorator añade responsabilidades (nuevo comportamiento). Proxy controla el acceso (no añade comportamiento, solo gestiona quién/cuándo puede acceder).
- **Decorator vs. Adapter**: Decorator mantiene la **misma interfaz** y añade funcionalidad. Adapter **cambia la interfaz** para hacer compatible algo incompatible.

4.4.6 Ventajas e inconvenientes

Ventajas:

- **Flexibilidad** mayor que la herencia: puedes combinar decoradores en tiempo de ejecución.
- **Principio de responsabilidad única**: cada decorador hace solo una cosa.

Inconvenientes:

- Da lugar a muchos **objetos pequeños** similares entre sí.
- Puede ser difícil de **depurar**: al encadenar muchos decoradores, seguir el flujo de ejecución se complica.

4.5 Proxy (Apoderado)

4.5.1 ¿Qué problema resuelve?

Un editor de documentos puede contener muchas imágenes. Cargar todas las imágenes al abrir el documento sería muy lento. La solución: cargar cada imagen solo cuando el usuario realmente necesita verla ("carga perezosa" o lazy loading).

Pero el cliente (el editor) usa objetos `IGrafico` y espera poder llamar a `dibujar()` en cualquier momento. Si sustituimos la imagen real por un "representante" que tiene la misma interfaz y carga la imagen real solo cuando es necesario, el editor no nota la diferencia.

Solución: Proxy

Proporciona un **sustituto o representante** de otro objeto para **controlar el acceso** a él. El Proxy tiene la misma interfaz que el objeto real, por lo que el cliente no sabe que está hablando con un Proxy.

4.5.2 Tipos de Proxy

- **Proxy virtual**: retrasa la creación de objetos costosos hasta que son necesarios (lazy loading). El ejemplo de las imágenes.
- **Proxy remoto**: representa localmente un objeto que está en otro servidor o espacio de direcciones.
- **Proxy de protección**: controla el acceso al objeto real según permisos del cliente.
- **Referencia inteligente**: hace operaciones adicionales al acceder al objeto (p.ej. conteo de referencias, logging).

4.5.3 Los participantes

Participantes de Proxy

- **Sujeto**: interfaz común para `SujetoReal` y `Proxy`. Permite usar el Proxy en cualquier lugar donde se usa el SujetoReal.
- **SujetoReal**: el objeto real representado. Hace el trabajo de verdad.
- **Proxy**: tiene una referencia al SujetoReal y puede controlarlo. Implementa la misma interfaz que el SujetoReal.

4.5.4 La estructura UML explicada

Estructura del patrón Proxy

```
                 <<interface>>
                   IGrafico
                   +dibujar()
                   +obtenerExtension()
                        ^
                        |
            -----------|-----------
           |                       |
         Imagen                ProxyImagen
         -imagenImp             -nombreFichero
         +dibujar()             -imagen: Imagen  (null al inicio)
         +obtenerExtension()    +dibujar()
         (SujetoReal)           +obtenerExtension()
                                    |    <<instantiate>>
                                    +----> Imagen (solo cuando necesario)

  ProxyImagen.dibujar():
    if (imagen == null) {
        imagen = cargar(nombreFichero);  // carga perezosa
    }
    imagen.dibujar();

  ProxyImagen.obtenerExtension():
    if (imagen == null) {
        return extension;  // usa dato local, sin cargar imagen
    }
    return imagen.obtenerExtension();
```

Cómo leer este diagrama:

1. `IGrafico` es la interfaz (Sujeto): tanto `Imagen` como `ProxyImagen` la implementan.
2. `Imagen` es el SujetoReal: carga la imagen del disco (operación costosa).
3. `ProxyImagen` es el Proxy: tiene `-imagen: Imagen` (referencia al real, inicialmente nula) y `-nombreFichero`.
4. Cuando se llama a `dibujar()`: si la imagen no está cargada (`imagen == null`), la carga. Luego delega en `imagen.dibujar()`.
5. Cuando se llama a `obtenerExtension()`: si no está cargada, devuelve la extensión guardada localmente (sin cargar la imagen completa).
6. El Editor (`EditorDocumentos`) trabaja con `IGrafico`: no sabe si es `Imagen` real o `ProxyImagen`.

4.5.5 Diferencias con Adapter y Decorator

Las tres diferencias críticas: Adapter, Proxy, Decorator

- **Adapter**: **cambia la interfaz**. El adaptado tiene una interfaz *distinta* al objetivo. El adaptador traduce entre las dos.
- **Proxy**: **misma interfaz, controla el acceso**. El proxy puede rechazar, retrasar o registrar peticiones, pero la interfaz es idéntica al real.
- **Decorator**: **misma interfaz, añade responsabilidades**. El decorador añade nuevo comportamiento (scroll, borde...) encadenando envolturas.

5 1 T10: Patrones de Comportamiento

¿Qué son los Patrones de Comportamiento?

Los patrones de comportamiento definen **cómo interactúan y se comunican los objetos** entre sí. Se centran en los algoritmos y la asignación de responsabilidades entre objetos.

5.1 Observer (Observador)

5.1.1 ¿Qué problema resuelve?

Tienes una subasta online. Cuando alguien hace una puja (el valor cambia), necesitas notificar a varios interesados: enviar un email de confirmación, enviar un mensaje de WhatsApp, actualizar una pantalla. El número de interesados puede cambiar: mañana pueden añadirse notificaciones por SMS.

Sin Observer, la clase `Subasta` tendría que conocer directamente a `Email`, `WhatsApp`, `SMS`... y llamarlos explícitamente. Si añades SMS, tienes que modificar `Subasta`. Acoplamiento fuerte.

Solución: Observer

El objeto que tiene el estado interesante (el **sujeto**) mantiene una lista de **observadores** suscritos. Cuando su estado cambia, notifica a todos sus observadores. Los observadores se suscriben y se dan de baja dinámicamente. El sujeto no sabe exactamente quiénes son sus observadores ni cuántos son. Establece una **dependencia uno-a-muchos**.

5.1.2 Cuándo usarlo

- Cuando un cambio en un objeto requiere modificar otros **sin saber cuántos son**.
- Cuando un objeto debe **notificar a otros sin hacer suposiciones** sobre quiénes son.

5.1.3 Los participantes

Participantes de Observer

- **Sujeto** (Observable): conoce a sus observadores (los almacena en una lista). Proporciona la interfaz para suscribirse y darse de baja.
- **Observador**: interfaz con el método `actualizar()` que el sujeto llama cuando cambia.
- **SujetoConcreto**: almacena el estado de interés. Notifica a sus observadores

cuando ese estado cambia. Proporciona métodos para obtener/establecer el estado.

- **ObservadorConcreto**: tiene referencia al SujetoConcreto para consultar su estado. Implementa `actualizar()` para sincronizarse.

5.1.4 La estructura UML explicada

Estructura del patrón Observer

```
        Observable (Sujeto)              <<interface>>
        +suscribir(o: IObservador)  -----> IObservador
        +quitar(o: IObservador)     1    * +actualizar()
        +notificar()                            ^
              ^                          -------|-------
              |                        |               |
          Subasta                Email           WhatsApp
          -valor                -subasta         -subasta
          +obtenerValor()      +enviarMail()    +enviarMsg()
          +pujar()             +actualizar()    +actualizar()
              |
              | notificar():
              |   para todo o en suscriptores:
              |     o.actualizar();

  Email.actualizar():
    [...] subasta.obtenerValor(); [...]  <- consulta el nuevo valor

  Subasta.pujar(nuevaOferta):
    if (nuevaOferta > valor) {
        valor = nuevaOferta;
        notificar();  <- notifica a todos
    }
```

Cómo leer este diagrama:

1. `Observable` es la clase base del Sujeto. Tiene la lista `-suscriptores` de `IObservador` (multiplicidad 1 a $*$). Implementa `suscribir()`, `quitar()` y `notificar()`.

2. `IObservador` es la interfaz de los observadores, con un único método `actualizar()`.

3. `Subasta` extiende `Observable` (hereda la gestión de suscriptores) y añade su estado (`-valor`) y operaciones (`pujar()`).

4. `Email` y `WhatsApp` implementan `IObservador` y tienen una referencia `-subasta` al `SujetoConcreto` para consultar el nuevo valor en `actualizar()`.

Flujo de ejecución completo

1. `Email email = new Email(subasta)` y `subasta.suscribir(email)`.
2. `WhatsApp wa = new WhatsApp(subasta)` y `subasta.suscribir(wa)`.
3. Alguien llama a `subasta.pujar(150)`.
4. Dentro de `pujar()`: si 150 > valor actual, `valor = 150` y se llama a `notificar()`.
5. `notificar()` itera: llama a `email.actualizar()` y `wa.actualizar()`.
6. `email.actualizar()` llama a `subasta.obtenerValor()` → obtiene 150 → envía el email.
7. `wa.actualizar()` llama a `subasta.obtenerValor()` → obtiene 150 → envía el WhatsApp.
8. Si mañana añadimos SMS: creamos `SMS` implementando `IObservador` y hacemos `subasta.suscribir(sms)`. No modificamos `Subasta`.

5.1.5 Ventajas e inconvenientes

Ventajas:

- **Acoplamiento mínimo**: el sujeto no conoce los tipos concretos de sus observadores.
- **Comunicación por difusión**: una notificación llega a todos automáticamente.
- **Modificación independiente**: puedes añadir/quitar observadores sin tocar el sujeto.

Inconvenientes:

- Posibles **actualizaciones en cascada** inesperadas: un observador actualiza algo que dispara otra notificación.
- El protocolo simple de `actualizar()` **no detalla qué cambió** en el sujeto (si hay muchos atributos, el observador no sabe cuál cambió).

5.2 State (Estado)

5.2.1 ¿Qué problema resuelve?

Una conexión TCP puede estar en varios estados: **Establecida**, **Escuchando** o **Cerrada**. Cada estado responde de forma diferente a las mismas operaciones (`abrir()`, `cerrar()`, `acuseDeRecibo()`).

Sin State, el código de `ConexionTCP` estaría lleno de condicionales:

```
public void abrir() {
    if (estado == ESTABLECIDA) {
        // hacer X
    } else if (estado == ESCUCHANDO) {
        // hacer Y
    } else if (estado == CERRADA) {
        // hacer Z
    }
}
// Lo mismo en cerrar(), acuseDeRecibo()...
```

Estos bloques son largos, difíciles de mantener y fáciles de romper al añadir nuevos estados.

Solución: State

Encapsula cada estado en su propia clase. El objeto contexto (`ConexionTCP`) delega las operaciones al objeto estado actual. Cuando el estado cambia, simplemente cambia el objeto estado al que delega. Parece que el objeto cambia de clase.

5.2.2 Cuándo usarlo

- Cuando el comportamiento de un objeto **depende de su estado** y debe cambiar en tiempo de ejecución.
- Cuando hay métodos con **condicionales largas** dependientes del estado del objeto.

5.2.3 Los participantes

Participantes de State

- **Contexto** (ej: `ConexionTCP`): define la interfaz de interés para los clientes. Mantiene una instancia de EstadoConcreto que representa el estado actual. Delega operaciones al objeto estado.
- **Estado** (ej: `IEstadoTCP`): interfaz que encapsula el comportamiento asociado a cada estado.
- **EstadoConcreto** (ej: `TCPEstablecida`, `TCPEscuchando`, `TCPCerrada`): implementa el comportamiento de un estado particular. Puede cambiar el estado del contexto.

5.2.4 La estructura UML explicada

Estructura del patrón State

```
        ConexionTCP (Contexto)          <<interface>>
        +abrir()                  -----> IEstadoTCP (Estado)
        +cerrar()                 -estado   +abrir()
        +acuseDeRecibo()                    +cerrar()
        +configurar()                       +acuseDeRecibo()
                                                 ^
                                             |
                                  -------------|------------
                                 |             |            |
                              TCPEstablecida TCPEscuchando TCPCerrada
                              +abrir()        +abrir()      +abrir()
                              +cerrar()       +cerrar()     +cerrar()
                              +acuseDeRecibo()...           ...

  ConexionTCP.abrir():
    estado.abrir();  <- delega al estado actual

  TCPCerrada.abrir():
    // hacer lo que corresponde cuando esta cerrada y se abre
    conexion.setEstado(new TCPEstablecida());  // cambia el estado
```

Cómo leer este diagrama:

1. `ConexionTCP` (Contexto) tiene una referencia `-estado` a `IEstadoTCP` (agregación con diamante en el diagrama).
2. Los métodos de `ConexionTCP` simplemente delegan: `estado.abrir()`, `estado.cerrar()`, etc.
3. `IEstadoTCP` es la interfaz Estado con los mismos métodos.
4. `TCPEstablecida`, `TCPEscuchando` y `TCPCerrada` implementan `IEstadoTCP` y contienen el comportamiento específico de cada estado.
5. Las transiciones de estado ocurren dentro de los estados concretos: cuando `TCPCerrada.abrir()` se ejecuta, puede cambiar el estado del contexto a `new TCPEstablecida()`.

Flujo de ejecución

1. Estado inicial: `conexion.estado = new TCPCerrada()`.
2. El cliente llama a `conexion.abrir()`.
3. `ConexionTCP.abrir()` delega: `estado.abrir()` → `TCPCerrada.abrir()`.
4. `TCPCerrada.abrir()` hace lo que corresponde y luego: `conexion.setEstado(new TCPEscuchando())`.
5. El cliente llama de nuevo a `conexion.abrir()`.
6. Ahora `estado` es `TCPEscuchando`, así que `TCPEscuchando.abrir()` se ejecuta (distinto comportamiento).

5.2.5 Implementación: ¿quién define las transiciones?

- **El Contexto**: si las transiciones son fijas y simples. El contexto decide qué estado sigue a cuál.
- **Los EstadosConcretos**: si las transiciones son complejas o dependen de cada estado. Más flexible pero más acoplado (los estados se conocen entre sí).

5.2.6 Ventajas e inconvenientes

Ventajas:

- Todo el comportamiento de un estado está **en un solo objeto**: fácil de encontrar y modificar.
- **Fácil añadir nuevos estados**: solo creas una nueva clase.
- **Elimina los condicionales** largos dependientes del estado.
- **Explicita las transiciones**: protege frente a estados inconsistentes.

Inconvenientes:

- **Incrementa el número de clases**: cada estado es una clase.

5.3 Strategy (Estrategia)

5.3.1 ¿Qué problema resuelve?

Tienes un generador de superficies (`GenSuperficies`) que puede interpolar datos usando tres algoritmos diferentes:

- **IDW**: rápido pero menos preciso.
- **Spline**: precisión media, velocidad media.
- **Kriging**: muy preciso pero lento.

Sin Strategy, `interpolar()` tendría un `switch` enorme. Cambiar de algoritmo requeriría modificar la clase. Añadir un nuevo algoritmo también.

Solución: Strategy

Encapsula cada algoritmo en su propia clase. El contexto (`GenSuperficies`) tiene una referencia a una **estrategia** (interfaz) y delega el algoritmo a ella. El cliente puede cambiar la estrategia en tiempo de ejecución sin modificar el contexto.

5.3.2 Cuándo usarlo

- Cuando existen varias clases relacionadas que **solo difieren en su comportamiento**.
- Cuando se necesitan **variantes de un algoritmo**.
- Cuando un algoritmo usa datos que los clientes **no necesitan conocer**.
- Para **eliminar condicionales largas** de selección de comportamiento.

5.3.3 Los participantes

Participantes de Strategy

- **Estrategia**: interfaz común a todos los algoritmos. El contexto la usa para ejecutar el algoritmo concreto.
- **EstrategiaConcreta**: implementa un algoritmo en particular.
- **Contexto**: mantiene una referencia a la Estrategia. Se configura con una estrategia concreta. Puede definir una interfaz para que la estrategia acceda a sus datos.

5.3.4 La estructura UML explicada

Estructura del patrón Strategy

```
     GenSuperficies (Contexto)        <<interface>>
     +especificarSalida()  ---------> IInterpolador (Estrategia)
     +interpolar()         -estrategia   +calcular()
                                              ^
                                   -----------|-----------
                                  |           |           |
                                IDW         Spline     Kriging
                               +calcular() +calcular() +calcular()

  GenSuperficies.interpolar():
    estrategia.calcular();  <- delega en la estrategia actual

  Uso:
    gen.setEstrategia(new IDW());    // rapido
    gen.interpolar();
    gen.setEstrategia(new Kriging()); // preciso
    gen.interpolar();
```

Cómo leer este diagrama:

1. `GenSuperficies` (Contexto) tiene una referencia `-estrategia` a `IInterpolador` (agregación con diamante).
2. `IInterpolador` es la interfaz Estrategia con el método `calcular()`.
3. `IDW`, `Spline` y `Kriging` son EstrategiasConcreta que implementan `IInterpolador`.
4. `GenSuperficies.interpolar()` simplemente llama a `estrategia.calcular()`.
5. El cliente puede cambiar la estrategia en cualquier momento con `gen.setEstrategia(new Kriging())`.

5.3.5 La diferencia crucial: State vs. Strategy

Estructuralmente, State y Strategy parecen idénticos (un contexto con referencia a una interfaz, múltiples implementaciones). La diferencia es **semántica**:

State vs. Strategy — La diferencia definitiva

azulclaro **Aspecto**	**State**	**Strategy**
¿Quién cambia?	El propio estado (interno)	El cliente (externo)
¿Los objetos se conocen?	Sí (los estados se conocen entre sí para las transiciones)	No (las estrategias son independientes)
Propósito	Variar comportamiento según estado interno	Intercambiar algoritmos
¿Cuándo cambia?	El estado decide cuándo transicionar	El cliente decide cuándo cambiar
Analogía	Un semáforo que cambia solo	Elegir qué ruta tomar en el GPS

Señal práctica: si los objetos de la interfaz se conocen entre sí o pueden autocambiarse -> es State. Si son independientes y el cliente los elige libremente -> es Strategy.

5.3.6 Ventajas e inconvenientes

Ventajas:

- Familia de **algoritmos intercambiables**.
- Permite **modificar cualquier algoritmo** con independencia del contexto.
- **Elimina condicionales** de selección de comportamiento.

Inconvenientes:

- Los clientes deben **conocer las estrategias** disponibles para elegir.
- Las estrategias simples pueden **no usar toda la información** que reciben del contexto.

5.4 Template Method (Método Plantilla)

5.4.1 ¿Qué problema resuelve?

Tienes un framework de aplicaciones. La clase `Aplicacion` define el flujo general para abrir documentos:

1. Comprobar si se puede abrir.
2. Crear el documento.
3. Añadirlo a la lista de documentos abiertos.
4. Notificar que va a abrirse.
5. Abrir el documento.
6. Leer su contenido.

Este flujo es siempre el mismo. Pero los pasos concretos (¿qué tipo de documento crear?) dependen de la aplicación específica. No puedes implementarlos en la clase base.

Solución: Template Method

La clase base define el **esqueleto del algoritmo** (los pasos y su orden) en un método concreto (el "método plantilla"). Los pasos variables se declaran como métodos abstractos. Las subclases implementan esos pasos variables sin alterar la estructura del algoritmo.

5.4.2 Cuándo usarlo

- Para implementar las **partes invariantes** de un algoritmo dejando las partes variables a las subclases.
- Cuando el **comportamiento repetido** debe factorizarse en una clase común (evitar duplicación de código).
- Para **controlar extensiones** de subclases mediante "operaciones de enganche".

5.4.3 Los participantes

Participantes de Template Method

- **ClaseAbstracta**: implementa el método plantilla (concreto, no abstracto) que define el esqueleto. Declara operaciones primitivas abstractas (que las subclases deben implementar) y operaciones de enganche (hooks, opcionales).
- **ClaseConcreta**: implementa las operaciones primitivas abstractas para realizar los pasos específicos.

5.4.4 La estructura UML explicada

Estructura del patrón Template Method

```
        ClaseAbstracta (Aplicacion)
        +abrirDoc(nombre)          <- metodo plantilla (CONCRETO, no abstracto)
        +crearDoc()                <- operacion primitiva (ABSTRACTA, cursiva)
        +sePuedeAbrirDoc()         <- hook (vacio por defecto, opcional)
        +aPuntoDeAbrirDoc()        <- hook (vacio por defecto, opcional)
             ^
             |  (herencia)
             |
        ClaseConcreta (MiAplicacion)
        +crearDoc()                <- implementa obligatoriamente
        +sePuedeAbrirDoc()         <- sobreescribe opcionalmente
        +aPuntoDeAbrirDoc()        <- sobreescribe opcionalmente

    abrirDoc(nombre):              <- el esqueleto del algoritmo
      if (!sePuedeAbrirDoc(nombre)) return;
      Documento doc = crearDoc();   <- llama a la primitiva abstracta
      if (doc != null) {
          docs.anadir(doc);
          aPuntoDeAbrirDoc(doc);    <- hook
          doc.abrir();
          doc.leer();
      }

    MiAplicacion.crearDoc():
      return new MiDocumento();
```

Cómo leer este diagrama:

1. `Aplicacion` (ClaseAbstracta) tiene el método `abrirDoc()` que es **concreto** (tiene implementación). Este es el método plantilla: define el flujo completo.

2. `crearDoc()` es **abstracto** (en cursiva en el diagrama): las subclases DEBEN implementarlo.

3. `sePuedeAbrirDoc()` y `aPuntoDeAbrirDoc()` son **hooks** (enganches): métodos vacíos por defecto que las subclases PUEDEN redefinir opcionalmente.

4. `MiAplicacion` (ClaseConcreta) hereda de `Aplicacion` e implementa `crearDoc()` devolviendo `new MiDocumento()`.

5. Cuando el cliente llama a `app.abrirDoc("archivo")`, se ejecuta el flujo completo definido en `Aplicacion`, pero en el paso de crear el documento, se invoca el `crearDoc()` de `MiAplicacion`.

5.4.5 Los tres tipos de operaciones en el método plantilla

azulclaro **Tipo**	**En ClaseAbstracta**	**En ClaseConcreta**
Primitiva abstracta	Declarada abstracta (cursiva)	DEBE implementarse (obligatorio)
Operación concreta	Implementada con comportamiento real	Puede redefinirse (opcional)
Hook (enganche)	Implementada vacía (no hace nada)	PUEDE redefinirse (opcional)

5.4.6 Template Method vs. Strategy

Template Method vs. Strategy

azulclaro **Aspecto**	**Template Method**	**Strategy**
Mecanismo	Herencia	Composición
Granularidad	Pasos individuales del algoritmo	El algoritmo completo
Variación	En compilación (subclase fija)	En ejecución (objeto intercambiable)
Quién varía	La subclase redefine pasos	El cliente elige la estrategia

Regla rápida: Template Method usa herencia para variar *partes* del algoritmo. Strategy usa composición para intercambiar el *algoritmo completo*.

5.4.7 Ventajas e inconvenientes

Ventajas:

- **Reutilización** del código del algoritmo: el flujo común está en un solo lugar.
- **Inversión de control**: el framework llama al código de las subclases ("no nos llames, te llamamos").

Inconvenientes:

- Puede ser difícil de mantener si hay **muchos pasos** en el algoritmo.
- Las subclases solo pueden variar **dentro del esqueleto** definido.

6 1 Tablas Resumen: Identificación Rápida

6.1 Señales visuales en UML por patrón

azulclaro **Patrón**	**Señal visual clave en el diagrama UML**
Abstract Factory	Interfaz fábrica con varios métodos `crearX()` + varias fábricas concretas + varias familias de productos
Factory Method	Clase abstracta con un único método abstracto que devuelve un objeto de su misma jerarquía
Adapter	Clase que implementa interfaz A Y contiene un objeto de interfaz B (distinta). Traducción de llamadas.
Bridge	Dos jerarquías paralelas separadas, conectadas por una referencia `-imp`
Composite	Clase que implementa interfaz C Y tiene colección de la misma interfaz C (auto-referencia recursiva)
Decorator	Clase abstracta que implementa interfaz C Y tiene referencia 1:1 a la misma interfaz C + añade comportamiento
Proxy	Dos clases implementan la misma interfaz; una tiene referencia a la otra y delega selectivamente
Observer	Clase con lista `-observadores` y método `notificar()` + interfaz Observador con `actualizar()`
State	Contexto con referencia `-estado` a interfaz abstracta + múltiples EstadosConcretos
Strategy	Contexto con referencia `-estrategia` a interfaz abstracta + múltiples EstrategiasConcreta
Template Method	Clase abstracta con método concreto que llama a métodos abstractos propios

6.2 Comparativa completa de todos los patrones

azulclaro **Patrón**	**Tipo**	**Idea clave**	**Cuándo usarlo**
Abstract Factory	Creación	Interfaz para crear familias de objetos	Familias de productos coherentes entre sí
Factory Method	Creación	Subclase decide qué instanciar	Clase no sabe qué tipo de objeto crear
Adapter	Estructural	Convierte interfaz incompatible	Reutilizar clase con interfaz diferente
Bridge	Estructural	Separa abstracción e implementación	Evitar explosión de subclases por dos dimensiones

Composite	Estructural	Árbol parte-todo uniforme	Jerarquías recursivas, tratar igual hoja y compuesto
Decorator	Estructural	Añade responsabilidades dinámicamente	Extensión flexible sin herencia
Proxy	Estructural	Sustituto para controlar acceso	Acceso remoto, lazy loading, protección
Observer	Comportamiento	Notificación automática 1:N	Cambio en un objeto debe notificar a varios
State	Comportamiento	Comportamiento según estado interno	Comportamiento depende del estado, eliminar switch
Strategy	Comportamiento	Algoritmos intercambiables	Variantes de un algoritmo, eliminar condicionales
Template Method	Comportamiento	Esqueleto de algoritmo en clase base	Flujo fijo con pasos variables

6.3 Los tríos confusos: Adapter, Proxy y Decorator

Cómo diferenciar Adapter, Proxy y Decorator

Los tres tienen una clase que "envuelve" a otra. La diferencia está en **para qué**:

azulclaro	**Adapter**	**Proxy**	**Decorator**
Interfaz al cliente	**Distinta** (nueva)	Idéntica al original	Idéntica al original
Propósito	Compatibilizar	Controlar acceso	Añadir responsabilidades
Encadenamiento	No	No	Sí (capas)
Señal UML	Implementa A, contiene B	Implementa A, contiene A	Implementa A, contiene A, añade antes/después

6.4 State vs. Strategy vs. Template Method

Cómo diferenciar State, Strategy y Template Method

Los tres son formas de variar un comportamiento:

azulclaro	**State**	**Strategy**	**Template Method**
Mecanismo	Composición	Composición	Herencia
¿Qué varía?	Comportamiento según estado	Algoritmo completo	Pasos del algoritmo
¿Quién cambia?	El estado (interno)	El cliente (externo)	La subclase
¿Cuándo?	En ejecución	En ejecución	En compilación
Los objetos se conocen	Sí (transiciones)	No	No aplica

6.5 Abstract Factory vs. Factory Method

Cómo diferenciar Abstract Factory y Factory Method

azulclaro **Aspecto**	**Factory Method**	**Abstract Factory**
Mecanismo	Herencia	Composición
¿Qué crea?	Un producto	Familia de productos
¿Quién decide?	La subclase (Creador-Concreto)	El objeto fábrica
Señal UML	Clase abstracta + método abstracto de creación	Interfaz fábrica con N métodos crearX()

7 1 Guía Práctica para el Examen

7.1 Método en 5 pasos para identificar un patrón

1. **Identifica el patrón**: busca en el diagrama UML las señales visuales características (ver tabla de la sección anterior).
2. **Nombra los participantes**: para cada clase del diagrama, di cuál es su rol en el patrón. Ejemplo: "`ConexionTCP` es el Contexto, `IEstadoTCP` es la interfaz Estado, `TCPEstablecida` es un EstadoConcreto".
3. **Explica la motivación**: ¿qué problema resuelve? ¿qué pasaría sin este patrón?
4. **Explica cómo funciona**: describe el flujo de colaboración entre participantes. "El Contexto delega en el estado actual. Cuando el estado cambia, se reemplaza el objeto estado."
5. **Indica ventajas e inconvenientes**: al menos 2 ventajas y 1 inconveniente.

7.2 Preguntas tipo que suelen aparecer

- Dado un diagrama UML, **identifica el patrón y nombra los participantes**.
- **Explica la diferencia** entre dos patrones (frecuentes: Decorator/Proxy, State/Strategy, Adapter/Bridge, Template Method/Strategy, Abstract Factory/Factory Method).
- **¿En qué situación usarías** el patrón X? Razona tu respuesta.
- **Indica ventajas e inconvenientes** del patrón X.
- Dado un problema de diseño, **¿qué patrón aplicarías**?

7.3 Participantes rápidos de cada patrón

azulclaro **Patrón**	**Participantes**
Abstract Factory	FabricaAbstracta, FabricaConcreta, ProductoAbstracto, ProductoConcreto, Cliente
Factory Method	Producto, ProductoConcreto, Creador, CreadorConcreto
Adapter	Objetivo, Cliente, Adaptado, Adaptador
Bridge	Abstraccion, AbstraccionRefinada, Implementador, ImplementadorConcreto
Composite	Componente, Hoja, Compuesto, Cliente

Decorator	Componente, ComponenteConcreto, Decorador, DecoradorConcreto
Proxy	Sujeto, SujetoReal, Proxy
Observer	Sujeto, Observador, SujetoConcreto, ObservadorConcreto
State	Contexto, Estado, EstadoConcreto
Strategy	Estrategia, EstrategiaConcreta, Contexto
Template Method	ClaseAbstracta, ClaseConcreta

Reflexión final

> *"Los mejores diseños combinan diversos patrones que se superponen para producir un mejor resultado."*
>
> — Gamma et al., *Patrones de Diseño*, 1994

Los patrones de diseño proporcionan un vocabulario común que eleva la abstracción, reduce la complejidad del sistema, y facilita la comunicación entre desarrolladores.